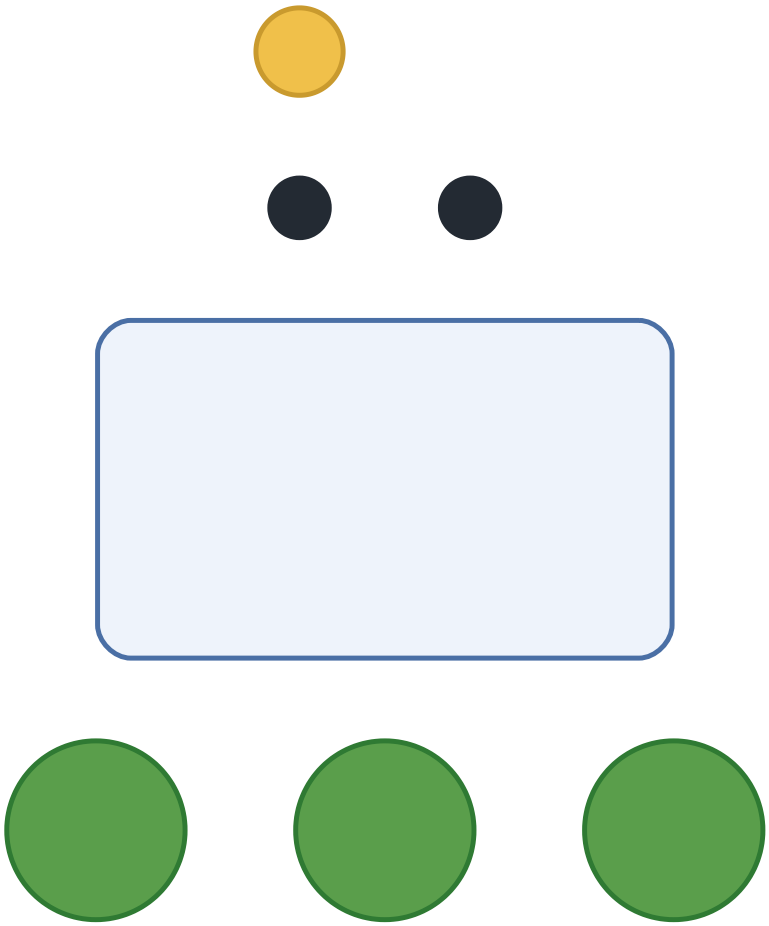
# Apitor Remote

## The big manual for young pilots

robots Robot R · J · S · Q · X · Wheels

for kids 6–15 and their grown-ups



source: docs/instructions-en.md · Russian version: instructions-ru.pdf

# Table of contents


1. What the remote is — 3
2. Meet the robot — 4
3. The first connection — 5
4. How to drive — 6
5. The light — 7
6. Three remote modes — 8
7. Your first program — 9
8. The “Square” program — 10
9. The “Firefly” program — 11
10. Challenges — 12
11. Program files and the editor — 13
12. The macro and the way back — 14
13. If something is wrong — 16
14. For grown-ups: install, flags, tuning — 17
15. Mini dictionary — 19
16. Cheat sheet — 20


**How to read this manual.** Keys are typed on the keyboard, so they always sit in a box: `q`, `w`, `Shift+Q`. Programs can be written with words — like this:

```
repeat 4
forward 4
right 2
```

A yellow box is a hint, a red one says “careful”, a green one is a challenge: try it yourself. Notes like “section 5, page 7” point to other pages. The ⋆ marks show how hard a task is: ⋆ easy, ⋆⋆ think a little, ⋆⋆⋆ for masters.

**Where to start:**

| If you want to… | Open |
|---|---|
| just drive around | sections 3–5 |
| teach the robot a program | sections 6–8 |
| put on a show with light | sections 9 and 10 |
| send the robot back along its tracks | section 12 |
| fix something that went wrong | section 13 |
| install and tune the remote (grown-ups) | section 14 |

# 1. What the remote is

The “remote” is a program on your computer. It talks to the Apitor robot over Bluetooth and tells it what to do. You press keys — the robot drives, spins and blinks its light.

What you need:

- an **Apitor robot** — built and with a charged battery;
- a **computer with Bluetooth** — a laptop, or a desktop with a Bluetooth adapter;
- a **terminal window** where the remote is already running.

When everything is ready, you see something like this:

```
[info] Apitor remote — Robot R (product 'r')
[hint] q — connect · Shift+Q — disconnect · w/a/s/d — drive · …
```

The remote is not connected to the robot yet — that happens when you press `q` (section 3, page 5). If there is no welcome line at all, ask a grown-up: section 14, page 17.

What the remote can do:

| What | How | Section |
|---|---|---|
| drive forward and back, spin around | `w` `s` `a` `d`, arrows | 4 |
| change the speed, switch the boost on | `]` `[`, `Shift` | 4 |
| light the LED up, blink, switch it off | `1` … `8`, `0`, `b` | 5 |
| spin motor 3 (an accessory) | `v` `V` | 2 |
| drive a program made of steps | `p`, then `Enter` | 6–8 |
| record a trip and drive it back | `m`, `r`, `t` | 12 |
| keep programs in files | `f`, `g`, `e` | 11 |

What the remote cannot do: it does not drive the robot from far away (stay within a few metres), it does not replace the official app, and it cannot carry the robot up the stairs.

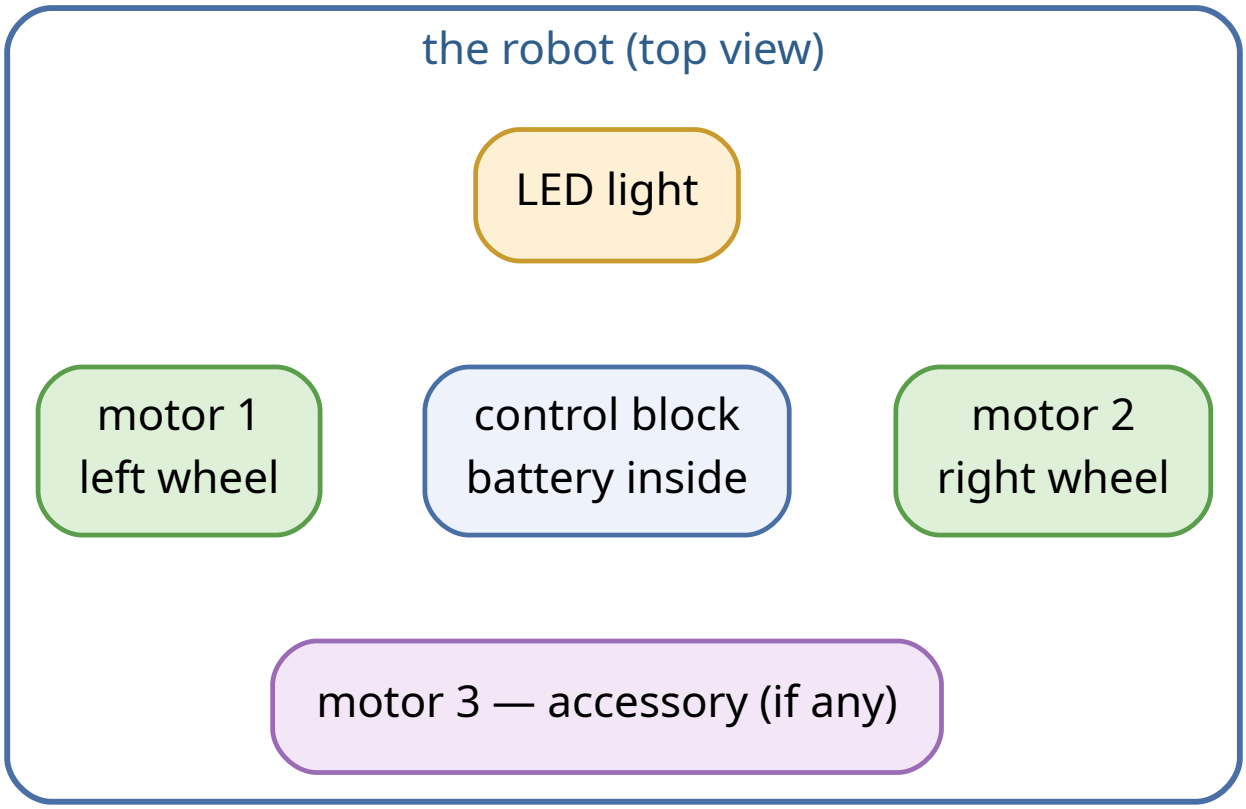
The remote also tells you what is going on: the lines `[robot]`, `[prog]`, `[run]` are its diary. They show what the robot is doing right now:

```
[robot] connected: Robot R (AA:BB:CC:DD:EE:FF)
[prog] ↑5 · →2 (2 steps · ~1.8 s · speed 6)
[run] done: 2 steps in 1.8 s (planned 1.8 s)
```

Robots come in different shapes: `Robot R`, `J`, `S`, `Q`, `X` are cars, `Wheels` is a wheeled base. Inside they are alike: two wheel motors, an LED light and a third motor for accessories — so the remote works with every model.

> **Challenge “First look”.** Open the remote and find three lines on the screen: `[robot]`, `[prog]` and `[run]`. Tell what each one means. Then find the `h` key — the remote will tell you about the keys itself.

# 2. Meet the robot




The robot seen from above: two wheels on the sides, the control block in the middle, the LED light in front, motor 3 for an accessory

- **Motor 1 and motor 2** — the left and the right wheel. When the wheels spin in opposite directions, the robot turns on the spot.
- **Control block** — the robot's "brain" and the battery. If the robot stays silent, the battery probably needs charging.
- **LED light** — a small lamp you can control from the keyboard (section 5, page 7).
- **Motor 3** — for extra mechanisms, if your model has them. Key `v` turns it forward, `V` turns it backward (it is a switch: press once to start, press again to stop).

Here are the same parts in one table:

| Robot part | What it is | Keys |
|---|---|---|
| Motor 1 and motor 2 | the left and the right wheel | `w` `s` `a` `d` |
| LED light | the lamp in front, seen from afar | `1` … `8`, `0`, `b` |
| Motor 3 | an axle for accessories (if you have them) | `v` `V` |
| Control block | the "brain" and the battery | — |

The robot drives "relative to itself": `w` goes wherever its nose points. Turn the robot nose away from you and everything becomes obvious.

> **Challenge "Robot exam".** Point at motor 1, motor 2, motor 3, the LED light and the control block. Remember which key turns each one on.

# 3. The first connection

1. Switch the robot on.
2. Put it on the floor — away from table edges, feet and pets.
3. Press `q`. The remote starts looking for the robot.
4. Wait a few seconds — you will see something like:

```
[robot] looking for the robot (powered on and not connected to a phone?)...
[robot] found Robot R (AA:BB:CC:DD:EE:FF), connecting...
[ok] connected: Robot R (AA:BB:CC:DD:EE:FF)
```

5. All set! The keys drive the robot now.

If the robot is connected to a phone, the computer cannot find it: Bluetooth talks to one device at a time. Close the phone app and press `q` again.

Next time `q` connects almost instantly: the robot's address is remembered. If the robot is switched off, the remote scans again and tells you about it.

| If something is wrong | What to do |
|---|---|
| It scans and never finds the robot | is the robot on? is the battery charged? |
| A phone has taken the robot | close the phone app and press `q` again |
| The wrong robot was found | `Shift+Q` and start with `--product` (section 14) |
| It is connected but will not drive | press `w` while the remote window is active; check `]` (speed) |
| The robot hums without moving | lower the speed: `[` |
| Want to check who is connected | the first line of the window: `[robot] connected: …` |

To disconnect, press `Shift+Q`. That is handy when the game is over: the robot stops and stops waiting for commands.

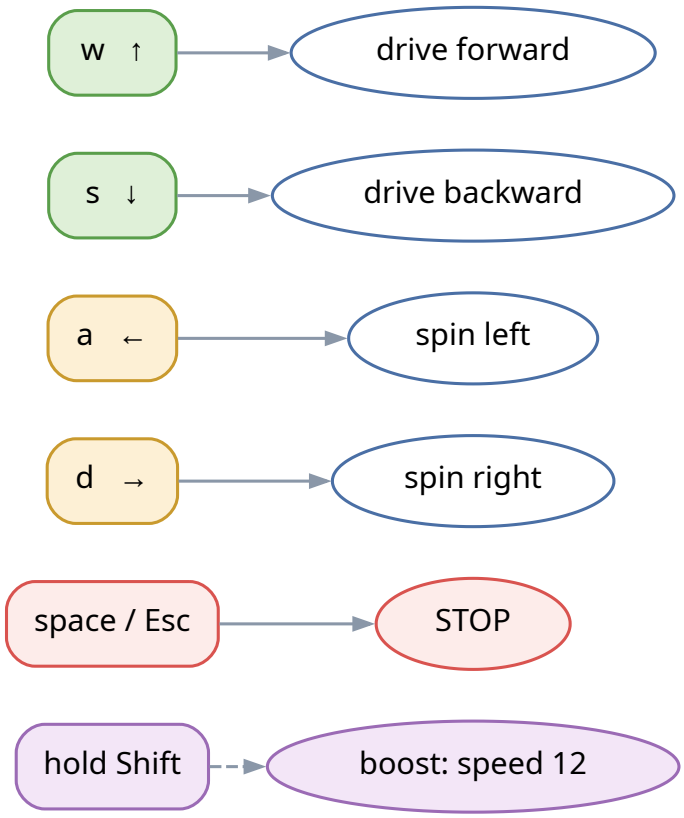
**Challenge "I can connect myself".** Switch the robot on and connect with `q` without help. Then disconnect (`Shift+Q`) and connect again: the second time is faster, because the robot's address is remembered.

**Why is the second time faster?** The remote remembers the robot's address in the file `addresses.json` (section 14, page 17), so `q` connects right away, without scanning. If the robot does not answer, the remote starts scanning again and refreshes the address.

If there are several robots around (or a friend has the same one), tell the remote which model you need: `--product r`, `--product j` — and so on up to `--product w`. And `--no-cache` makes it scan from scratch every time.

**How long to wait?** Usually two or three seconds. If fifteen have passed and there is no robot yet, press `q` again: the remote will start a new search. The battery lasts a long time, but before a long game it is best to put the robot on charge.

# 4. How to drive




Driving keys: w — forward, s — backward, a — left, d — right, space — stop, Shift — boost

| Keys | What they do |
|---|---|
| `w` or `↑` | forward |
| `s` or `↓` | backward |
| `a` or `←` | spin left on the spot |
| `d` or `→` | spin right on the spot |
| `space` or `Esc` | STOP — the robot freezes instantly |
| `]` `[` or `+` `-` | speed: 1 (very slow) … 12 (fast) |
| `Shift` (hold) | boost: speed 12 while held, the old speed comes back after |
| `j` or `k` | how to drive: “hold” or “cruise control” |

There are two ways to drive, switched with `j` and `k`:

- **hold**: the robot drives while the key is held down. Let go — it stops. This is the best way to learn and to park.
- **cruise control**: like in a car. Press `w` once and the robot drives on by itself, hands free; `space` is the brake — press it and the robot stops. Great for long straights and one-step programs.

> The `Shift` boost works only when the remote can watch real key presses (on Linux that needs a permission, see section 14, page 17). If boost does not come on, just drive at the usual speed. And remember: `Shift+Q` disconnects the robot — that is not the boost.

> **Challenge “Warm-up”.** Put the robot in the middle of the room. Drive forward for two counts, stop, come back. Then spin a quarter turn ( `a` or `d` ). Done? You are ready for section 7, page 9.

# 5. The light




Light colours: keys 1–8, key 0 switches the light off

| Key | Colour | Key | Colour |
|---|---|---|---|
| `1` | red | `5` | cyan |
| `2` | orange | `6` | blue |
| `3` | yellow | `7` | purple |
| `4` | green | `8` | white |

- `0` — switch the light off entirely.
- `b` or `B` — blink: the LED flashes quickly in the current colour. The colour itself does not change.

Colours can be written into a program as well: `color 1` is red, `color 5` is cyan, `color 0` switches the light off. In program mode the key `c` walks through the colours, and `c` plus a digit picks an exact one: `c3` is yellow.

> **Game “Traffic light”.** One player is the traffic light: yellow, red, green. The second player is the driver: stop on red, drive on green, get ready on yellow. Then swap.

> **Challenge “Garland”.** In program mode collect: `color 1`, `space`, `color 4`, `space`, `color 3`, `space` — and press `Enter`. The robot stands still and blinks three colours, like a garland on a tree.

For example, this little traffic-light program:

```
color 1
pause 1
color 4
pause 1
color 0
```

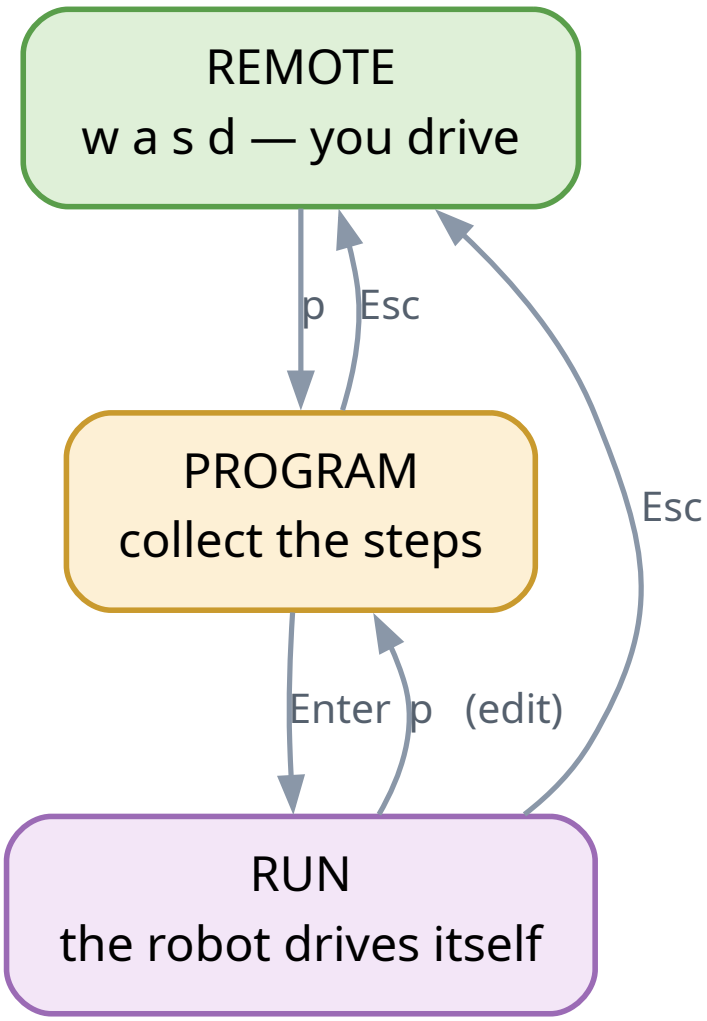
The robot stands in red, then in green, and then switches the light off — and you can drive on. The colour stays on until you press another one, just like a real toy car with a flashing light.

Where to press what — to avoid confusion:

| I want to | In the remote | In a program |
|---|---|---|
| light a colour | digits `1` … `8` | the step `color 4` or `c4` |
| switch the light off | `0` | the step `color 0` |
| blink | `b` or `B` | the step `blink` (or `b`) |
| walk through the colours | — | the `c` key |

> A blink does not change the colour: `b` flashes whatever is on right now. And digits in program mode do not light anything up — they land in a step: `c3` means “light up yellow”, not “light up now”.

# 6. Three remote modes




The three modes: remote, program, run — and the keys that move you between them

- **Remote** — the keys drive the robot (sections 4 and 5, pages 6 and 7).
- **Program** — you collect the steps; the robot stands still and waits.
- **Run** — the robot performs the program by itself.

How to move around:

| Keys | Where you get |
|---|---|
| `p` or `P` | into the program mode — always, from anywhere |
| `Enter` | start the program (the run begins) |
| `Esc` | back to the remote (the program stays in memory) |
| `Enter` | drive the program (with `Shift` — without pauses) |
| `r` `t` | in the remote: drive the route back (section 12, page 14) |

A trick: if you press `p` while the robot is driving, the remote stops it and opens the program. You can leave the program mode only with `Esc` — pressing `p` inside the program does nothing. No accidental way out.

During a run: `space` pauses and resumes, `Esc` stops and returns to the remote, `p` stops and goes straight to editing the program.

The LED shows the state: one green flash — start, two green flashes — finish, red — the program was interrupted. After the finish the previous colour comes back.

# 7. Your first program

A program is a list of steps: drive, turn, wait, blink, change the colour. You collect them in the program mode (press `p`).

| Keys | A program step |
|---|---|
| `w` `s` `a` `d` (or arrows) | forward, backward, left, right |
| digits `1` … `9` `0` | how many times: `wwwww` = “forward 5”, `w5` as well, `0` means ten |
| `space` | pause |
| `b` | blink |
| `c` | next colour; `c` plus a digit picks an exact colour (`c3` — yellow) |
| `]` `[` | program speed (digits right after: `]12`) |
| `*` (star, `Shift`+`8`) | repeat the whole program: `*` cycles 1→2→…, `*5` runs it 5 times |
| `Backspace` | delete the last step |
| `x` | clear the whole program |
| `Enter` | start it |
| `Esc` | back to the remote |

The keys `r` `R` `t` `T` work in this mode too: the robot drives the route back, and the program stays where it is (section 12).

Under the prompt the remote shows your program as symbols:

```
[prog] ↑5 · →1 · ✦2 (3 steps · ~4.2 s · speed 6)
```

The same symbols are used in files and in these diagrams:

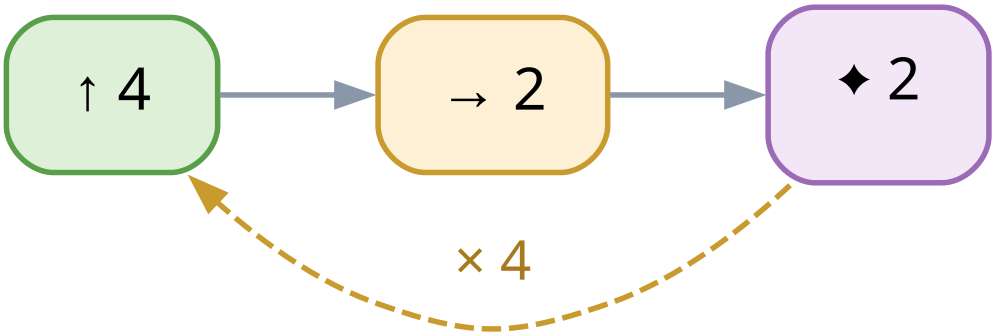
| Symbol | Step | Word |
|---|---|---|
| ↑ | forward | `forward` or `вперёд` |
| ↓ | backward | `back` or `назад` |
| ← → | left, right | `left` `right` |
| ‖ | pause | `pause` or `пауза` |
| ✦ | blink | `blink` or `мигнуть` |
| ● | colour | `color 3` or `цвет 3` |

The number next to a symbol is how many times the step repeats.

**How long it takes.** One “forward” step is about 0.6 seconds (the number is multiplied by 0.6), a turn is the same, a pause is 1 second and one blink is 0.25 seconds. The remote shows an estimate right away (`~4.2 s`) and the real time after the run.

> **Challenge “Knock-knock”.** Program: `forward 2`, `pause 1`, `back 2`. What happened? What if the pause becomes a blink — can the robot “knock” with light?

# 8. The “Square” program




Square: forward, turn, blink — four times around

The square is the first real program. The robot drives forward, turns, and repeats that four times — and you get a square. Collect it in the program mode:

```
w4      forward 4
d2      spin right 2
b2      blink 2
*4      repeat everything four times
```

Press `Enter` — and away it goes. The key `*` (the star) repeats the whole program, and the digit after it says how many times: `*4` means four laps. The very same program can be written with words in a program file (section 11, page 13):

```
repeat 4
forward 4
right 2
blink 2
```

A quarter turn is about `d2`. If your square comes out lopsided, try `d1` or `d3`: on different floors the robot turns a little differently. That is normal — the robot does not know whether it is on carpet or tiles.

**Challenge “Big square”.** Make a square twice as large: `forward 8`, `right 2`, `repeat 4`. Guess how many seconds the run will take and check yourself against the `[prog]` line.

**Why is it a square?** The turn `right 2` is about a quarter circle, 90°. Four turns together make 360°, a full circle, so the robot comes back to where it started. Put `right 1` and the robot turns only an eighth — and the square turns into a crooked heptagon. Maths!

**Challenge “Rectangle”.** Make two sides long and two short: `forward 8`, `right 2`, `forward 4`, `right 2` and `*2` at the end. Explain why repeating twice was enough.

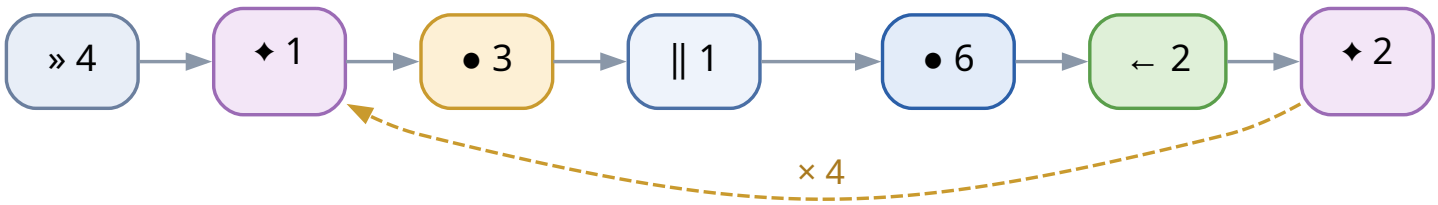
The `[prog]` line shows the length of the program: `(3 steps · ~4.2 s)`. So four laps take about sixteen seconds — just enough time to count to sixteen and check on the robot.

# 9. The “Firefly” program

Now a dance with light: the robot spins while the LED changes colour. Collect the program (or write it in a file and load it with `g`):

```
speed 4
repeat 4
blink 1
color 3
pause 1
color 6
pause 1
left 2
blink 2
```

What happens: the robot blinks yellow, waits, flashes blue, waits, spins left and blinks once more — four times around. A little show.




The “Firefly” scenario: blink, yellow, pause, blue, turn, blink — and four laps

| Step | What the robot does |
|---|---|
| `speed 4` | drives slower: the dance comes out smooth |
| `repeat 4` | runs the whole program four times |
| `blink 1` | flashes in the current colour |
| `color 3` | lights up yellow |
| `pause 1` | stands for a second |
| `color 6` | lights up blue |
| `left 2` | spins on the spot |
| `blink 2` | blinks twice |

**Challenge “Your own firefly”.** Change the colours and pauses to your own and make a show of four laps. Try `speed 8` and then `speed 2` — the dance comes out completely different.

A colour change is a step too: the robot changes the light not immediately, but when it reaches that step. That is why a “dance” of colours and pauses looks like a real light show.

# 10. Challenges

Stars show the difficulty: ⋆ — easy, ⋆⋆ — think a little, ⋆⋆⋆ — for masters.

1. ⋆ **The straight line.** Drive from one edge of the carpet to the other and come back.
2. ⋆ **Traffic light.** Program: red, pause, green, blink, drive.
3. ⋆⋆ **The snake.** Forward, turn right, forward, turn left — three times.
4. ⋆⋆ **Two squares.** The square from section 8 (page 10) with `repeat 2` — the robot must come back to where it started.
5. ⋆⋆ **The dance.** Spin, blink and change colours so that the grown-ups can guess the tune.
6. ⋆⋆⋆ **Boost race.** A long straight at speed 12 (the `Shift` boost) — without hitting the target.
7. ⋆⋆⋆ **The staircase.** Forward, right, forward, right… Build a path of steps to your toy. How many steps does it take?

Hints (open them only if you are really stuck):

8. ⋆⋆ **The figure of eight.** Two circles in opposite directions — an infinity sign.
9. ⋆⋆ **The patrol.** Drive around the room and come back to the same spot with the key `r`.
10. ⋆⋆⋆ **The repeater.** Drive around, record the trip as a macro (`m`, stop with `Esc`) and send the robot home with `r`.

Hints (open them only if you are really stuck):

- The snake: `w3 d1 w3 a1` — and repeat a few times.
- Two squares: put `repeat 2` at the very beginning of the program.
- The staircase: tune the turn number until you get a neat quarter turn (start with `d2`).
- The race: accelerate on the straight and press `space` before the target.
- The figure of eight: a circle left (`w2 a2`), a circle right (`w2 d2`) — and repeat.
- The patrol: drive back the way you came — or just press `r`.

**Challenge diary.** Write down what worked (and when):

| No. | Challenge | Done |
|---|---|---|
| 1 | The straight line | ☐ |
| 2 | Traffic light | ☐ |
| 3 | The snake | ☐ |
| 4 | Two squares | ☐ |
| 5 | The dance | ☐ |
| 6 | The boosted race | ☐ |
| 7 | The staircase | ☐ |
| 8 | The figure of eight | ☐ |
| 9 | The patrol | ☐ |
| 10 | The repeater | ☐ |

# 11. Program files and the editor

You can save a program into a file and load it next time.

| Keys | Action (in the program mode) |
|---|---|
| `f` | save: the remote asks for a name, `Enter` accepts it |
| `g` | load: a list of files, type a number and `Enter` |
| `e` (`E`) | edit the program in a full-size editor (works from the remote too) |
| `x` | clear the program |

Program files live in the `programs/` folder next to the remote. You can open them in a text editor and write with words — in English or in Russian:

```
# square: four sides and a blink at the end
speed 5
repeat 4
forward 4
right 2
blink 2
```

Every word the remote understands:

| Word | Symbol | Meaning |
|---|---|---|
| `forward`, `вперёд`, `up` | ↑ | drive forward |
| `back`, `назад`, `down` | ↓ | drive backward |
| `left`, `влево` | ← | turn left |
| `right`, `вправо`, `rght` | → | turn right |
| `pause`, `пауза` | ‖ | stand still |
| `blink`, `мигнуть` | ✦ | blink |
| `color 3`, `цвет 3` | ●3 | change the colour |
| `repeat 4`, `повтор 4` | — | run everything four times |
| `speed 5`, `скорость 5` | »5 | change the speed from here |
| `forward 1.2s`, `вперёд 1.2с` | ↑1.2s | drive for exactly 1.2 seconds |

Anything after `#` is a comment: the remote does not read it. Big and small letters, `ё` and `е`, commas and `×` are all forgiven. And it understands English and Russian words at once.

**Challenge “Out and back by file”.** Collect the square, save it with `f` as `square2.txt`, clear it (`x`) and load it again with `g`. Then press `e` and add the line `color 5` at the very top.

The program you collected will not be lost even if you close the remote: it keeps a draft and picks it up next time. Ready-made examples live in the `examples/` folder.

# 12. The macro and the way back

You do not have to collect a program by hand — you can **show** it: let the remote record your trip. Such a recording is called a **macro**.

**How to record:**

1. Press `m` — the log shows `[rec] recording a macro`.
2. Drive the robot as usual: `w` `a` `s` `d`, the light, the turns, the pauses. The remote records everything, and every ride becomes a step with seconds.
3. Press `Esc` (or `n`) — the recording is ready and already sits in the program line. `m` only starts the recording: pressing it again just reminds you that the recording is on.

```
[rec] recording a macro: driving, light, speed are all recorded, pauses become waits; Esc or n — stop
[rec] ↑
[rec] ←
[rec] macro recorded: 3 steps · 2.88 s — Enter — drive, r/t — back, f — save, p — edit
```

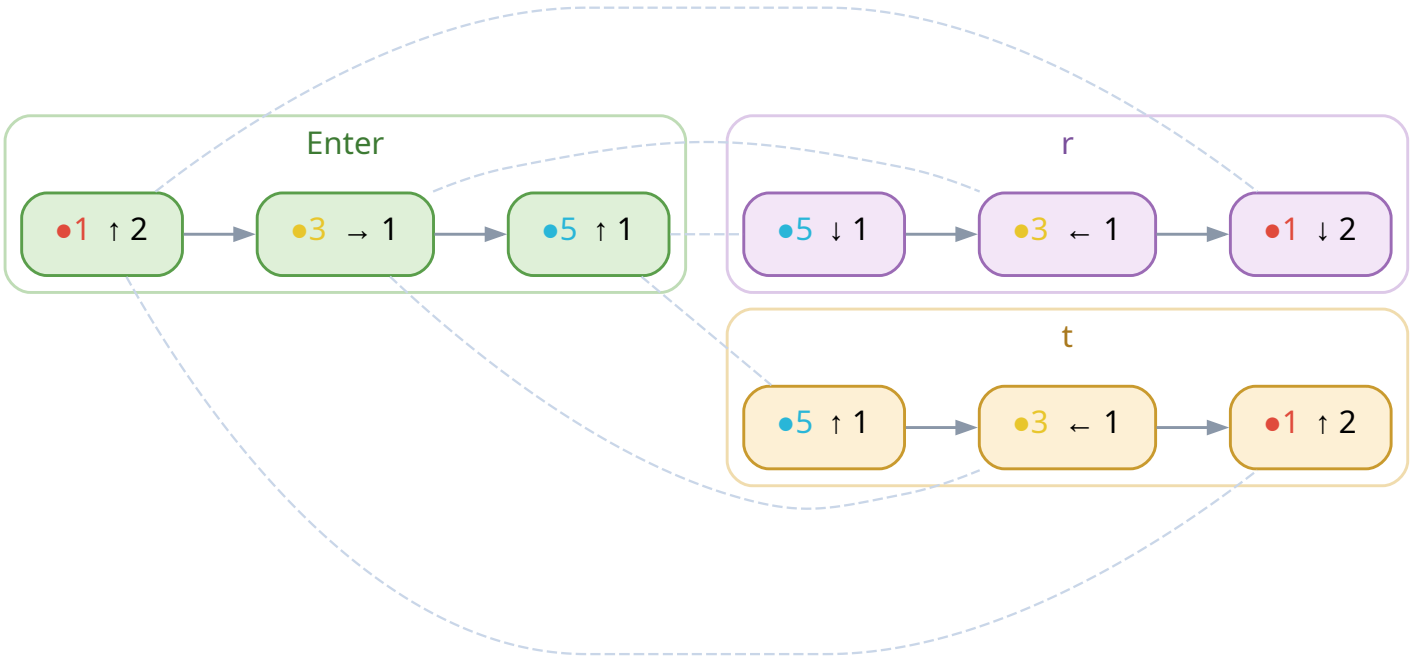
Now `Enter` — and the robot repeats the trip by itself: with the same pauses, light and speed. A macro is an ordinary program: you can edit it (`p`), save it into a file (`f`) and load it next time (`g`).

> While the recording is on, `p`, `Enter`, `r`, `t`, `x`, `f`, `g` do not work: finish the recording first — `Esc` or `n`. And `space` is just a robot stop: the recording continues.

## The way back: `r` and `t`

The robot drove away to the other end of the room. How to bring it back along its own tracks without collecting a second program? There are two keys for that:

- `r` — **backwards**. The robot drives the route from the end, and everything is mirrored: “forward” becomes “backward”, “left” becomes “right”. It backs along its own tracks.
- `t` — **the robot was turned by hand**. At the end of the route you turned it 180°, so it drives back forward: only the turns change (left ↔ right).




The way back: the same route from the end — backwards and forward

The duration is kept: if the robot drove for 2 seconds on the way out, it drives for 2 seconds on the way back as well (numbers like `↑5` become seconds).

**The speed and the colour switch exactly where you changed them.** You drive at 6 first, then switch to 12 and finish the route — on the way back the robot starts at 12 right away (that stretch was driven at 12), and at the turning point it

switches back to 6 and keeps it to the end. The colour works the same way: a stretch driven in cyan is driven back in cyan, and the colour switches exactly when you reach that stretch (in the picture `●5`, `●3`, `●1` stand in a different order). Stretches that ran "with whatever was lit" get the colour that was on before the run.

> `Shift+Enter` drives the program without pauses, and `R` or `T` (`Shift` with `r` or `t`) does the same on the way back. Capitals come by themselves, and `Shift+Enter` is learned from the terminal itself (kitty, foot, alacritty and others) — or from the keyboard tracking, when it is on (a grown-up knows what that is: section 14, page 17).
>
> The way back works from program mode too: type the steps, press `r` (or `t`) — the robot drives, and the program stays where it is.

The way back does not spoil the program: it is built on the fly, so after it `Enter` drives forward again and `p` shows the old steps. And `repeat` has nothing to do with it: back the robot always drives once — after the first pass it already stands at the start of the route and looks the other way.

> **Challenge "Out and back".** Drive a route with two speeds and two colours, record it as a macro and come back with `r`. Look at the `[run]` lines: you can see the robot started the way back at the last speed.

All four keys side by side:

| Keys | What the robot does |
|---|---|
| `r` | drives the route back, backwards |
| `t` | the same, but forward — for a robot you turned by hand |
| `R` `T` | the same without pauses, in one go |

In the log you can see it in the `[run]` lines:

```
[run] start: 9 steps · speed 6 · reversed (driving backwards)
[run] ▶ »12
[run] ▶ ↓0.6s
[run] ▶ ↓0.6s
[run] ▶ ↑0.6s
[run] ▶ »6
[run] ▶ ←0.6s
[run] done: 9 steps in 4.6 s
```

Speed 12 stands at the very beginning for a reason: it was put exactly where it changed in the original program.

While the recording is running the window shows lines like these:

```
[rec] recording a macro: driving, light, speed are all recorded,
      pauses become waits; Esc or n — stop
[rec] … do something (Esc or n — stop)
```

And when you stop the recording — how many steps you collected:

```
[rec] macro recorded: 7 steps · 5.4 s — Enter — drive, r/t — back,
      f — save, p — edit
```

**What to try with the way back:**

- The letter "U": forward, turn, forward, turn, forward — then press `r` and the robot drives the letter backwards by itself.
- A traffic-light track: write `color 1` at the start and `color 5` at the end of the route — on the way back the colours come in a different order.
- A long trip: write `speed 12` in the middle of the file — on the way back the robot drives fast first and comes home slowly.

# 13. If something is wrong

| What happened | What to do |
|---|---|
| `[err] could not connect` | is the robot on? not busy with a phone? press `q` again |
| The robot drives backward instead of forward | it is a setting: section 14 (page 17), “calibration” |
| The robot is too slow or too fast | `]` and `[` change the speed (1…12) |
| The keys do nothing | the remote listens to the keyboard only while its window is active; press `h` for a hint. Do not type `w a s d` in other windows |
| `Enter` does not start the program | look at the `[prog]` line: `0 steps` means the program is empty |
| The robot stopped in the middle of a program | `space` was pressed (pause) — press it again |
| Everything is frozen | `Ctrl+C`: the remote stops the motors and quits |
| Not sure what to press | `h` — help; `Esc` — back to the remote |
| `[err] the program was not changed` | there is a typo in the file: press `e` and look |
| A file is missing in the `g` list | check the `programs/` folder (section 14, page 17) |
| The robot blinks and does not drive | `space` was pressed — press it again |
| The light is different after a program | that is on purpose: the old colour comes back after the finish |
| The program ran in a strange way | press `p` and look at the steps; `x` clears them |
| The robot drove under the sofa | `Esc` — stop; then reach it by hand or reconnect with `q` |
| The program goes on forever | check `repeat`: the number at the start of the program |
| The light blinks by itself | that is a blink step (✦) from the program — wait or press `Esc` |

**First aid, step by step.** If the robot behaves strangely:

1. Press `Esc` — the remote stops the robot. This is safe at any moment.
2. Look at the last line in the window: it says what happened.
3. Press `h` for help, `q` to reconnect. The run journal is switched on with the `--log` flag (section 14).
4. If nothing helps at all: `Ctrl+C` and start the remote again.

**If the robot has driven far away.** Press `Esc` — the remote stops it instantly, wherever it is. Then there are three ways to bring the robot home: drive it yourself (just `w` in the right direction), press `r` (the robot returns along its tracks) or `t` if you have managed to turn it by hand. And if the robot has left the room altogether — `q` to reconnect and `y` to check the motors.

> Almost everything is cured by three things: `Esc`, `q` and `Ctrl+C`. Press them boldly — the robot will not break, and the remote always stops the motors before quitting.

# 14. For grown-ups: install, flags, tuning

**The easiest way:** double-click `run.sh` (Linux) or `run.bat` (Windows). The file creates `.venv`, installs the dependency, opens a terminal and starts the remote. Flags are passed through as they are: `run.sh --tui` or `run.bat --program examples/square.txt`.

**Install by hand** (once):

```
cd Apitor-WASD
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

**Run:**

```
./run.sh                                        # all of that in one command
.venv/bin/python apitor_remote.py               # the usual remote
.venv/bin/python apitor_remote.py --tui         # full-screen interface:
                                                # the banner on top shows the mode keys
.venv/bin/python apitor_remote.py --simulate    # no robot: print frames
.venv/bin/python apitor_remote.py --program examples/square.txt
```

On Linux the remote needs permission to read the keyboard (otherwise it warns and falls back to auto-stop): `sudo usermod -aG input "$USER"`, then log out and in again. On Windows Python 3.10+ and Bluetooth are enough.

The boost (hold `Shift` — speed 12) works only when the keyboard tracking is on. `Shift+Enter` and capitals `R`, `T` need no tracking: on start the remote asks the terminal whether it can tell `Shift` apart (the kitty keyboard protocol: kitty, foot, alacritty, wezterm, ghostty, a recent GNOME Terminal) and turns the extended keys on if it can. If you do not need the tracking at all (the auto-stop by `--hold-timeout` is enough), run with `--no-key-state`: the remote will not read the keyboard in `/dev/input` and will not warn.

Useful flags (the full list is in the README):

| Flag | Why |
|---|---|
| `--speed 6` | starting speed 1…12 |
| `--product r` | robot model: `r`, `j`, `s`, `q`, `x`, `w` |
| `--tui` | full-screen interface with a banner and the journal |
| `--no-cache` | do not use the saved robot address |
| `--simulate` | no robot: print the frames that would be sent |
| `--cruise` | start with cruise control on (the old name `--latched` still works) |
| `--program-dir DIR` | where program files live (default `programs/`) |
| `--step-time` `--turn-time` `--pause-time` `--blink-time` | step durations, s (0.6 / 0.6 / 1.0 / 0.25) |
| `--max-steps N` | limit the program length (0 — no limit) |
| `--log` | run journal in `~/.config/apitor-control/runs.log` |
| `--key-state-device PATH` | watch a single keyboard only |
| `--no-key-state` | do not read the keyboard at all: no permissions, no warnings |
| `--verbose` | detailed output: protocol frames and scanning |
| `--lang ru` | Russian interface |

**Calibration** (if the robot drives the wrong way). Connect and press `y`: the robot drives forward, backward and sideways, so you can see what is mixed up. Then start with the right flag:

```
.venv/bin/python apitor_remote.py --flip-left     # the left wheel spins the wrong way
.venv/bin/python apitor_remote.py --flip-right    # the right wheel
.venv/bin/python apitor_remote.py --left-motor 2 --right-motor 1   # the wheels are swapped
```

Key `i` runs the motors one by one — that shows which port drives which wheel.

**This manual.** Sources: `docs/instructions-ru.md`, `docs/instructions-en.md`; diagrams: `docs/diagrams/*.dot`. To rebuild the pictures, the HTML and the PDFs:

```
docs/build.sh
```

It needs `pandoc`, `graphviz`, `calibre` and `pdftotext` (poppler) — see `docs/README.md`.

A check without a robot (handy right after the install):

```
.venv/bin/python apitor_remote.py --simulate
```

The remote prints the frames it would send to the robot — you can see what would happen, and no robot is needed.

**FAQ.**

| Question | Answer |
|---|---|
| The robot is never found | is it on? charged? is a phone holding it? is Bluetooth on? |
| The remote warns about the keyboard | add the user to the `input` group, or start with `--no-key-state` |
| Timer stops are not wanted | enable keyboard tracking (the `input` group) or lower `--hold-timeout` |
| Where is the run journal | `~/.config/apitor-control/runs.log` (flag `--log`) |
| How to start from scratch | delete the files in `~/.config/apitor-control/` — the remote recreates them |
| Starting without a terminal | double-click `run.sh` / `run.bat` |
| Where to take example programs | the `examples/` folder, for instance `examples/square.txt` |

All the settings live in the folder `~/.config/apitor-control/`:

| File | What is inside |
|---|---|
| `addresses.json` | robot addresses: so that `q` connects at once |
| `config.json` | language, step durations, program folder, journal |
| `runs.log` | the run journal (switched on with `--log`) |

Deleting these files is the easiest way to “reset” the remote to factory settings: it recreates them on the next launch.

# 15. Mini dictionary

- **Remote** — the program that drives the robot from your keyboard (section 1, page 3).
- **Macro** — a recording of your trip (`m`) that becomes a program (section 12, page 14).
- **The way back** — the robot drives the same route from the end (`r`, `t`, section 12, page 14).
- **Step** — one action of a program: drive, turn, blink (section 7, page 9).
- **Program** — a list of steps the robot performs by itself (section 7, page 9).
- **Run** — performing the program from start to finish (section 6, page 8).
- **Repeat** — how many times the whole program runs (`*` or `repeat`, section 8, page 10).
- **Hold** — the mode where the robot drives while you hold the key (section 4, page 6).
- **Latched** — the mode where the robot drives until you press `space` (section 4, page 6).
- **Boost** — speeding up to 12 while `Shift` is held (section 14, page 17).
- **Draft** — the program the remote remembers between launches (section 11, page 13).
- **Terminal** — the window where the remote lives; keys are pressed in it (section 1, page 3).
- **Speed** — a number from 1 (very slow) to 12 (fast); `]` and `[` change it (section 4, page 6).
- **The light** — the robot's LED; colours `1`…`8`, and `0` switches it off (section 5, page 7).
- **Motor 3** — the third axle for accessories; `v` starts it, `V` stops it (section 2, page 4).
- **Symbol** — a short way to write a step: `↑5`, `‖`, `✦2`, `●3` (section 7, page 9).
- **Program file** — a text file with steps in the `programs/` folder (section 11, page 13).
- **Comment** — a note after `#`; the remote does not read it (section 11, page 13).
- **Second** — write a number with a letter: `1.2s` instead of "some steps" (section 7, page 9).
- **Challenge** — a task with ⋆ marks; there are ten of them (section 10, page 12).
- **Way back** — the same route from the end (`r`, `t`, section 12, page 14).
- **Recording** — a macro runs until you press `Esc` or `n` (section 12, page 14).
- **Editor** — program mode, where the steps are typed with keys (section 7, page 9).
- **Pause** — a step where the robot stands still and does nothing (section 7, page 9).
- **Cache** — the saved robot address: that is why `q` finds it at once (section 3, page 5).
- **Frame** — a command the remote sends to the robot over Bluetooth (section 14, page 17).
- **Model** — which robot this is: R, J, S, Q, X or Wheels (section 1, page 3).
- **Journal** — the records of runs; switched on with `--log` (section 14, page 17).
- **The square** — the first real program: four turns (section 8, page 10).
- **The firefly** — a dance program: turns and coloured flashes (section 9, page 11).
- **Vocabulary** — every word a program file understands (section 11, page 13).
- **Flag** — a word after the program name on the command line: `--tui`, `--simulate` (section 14, page 17).
- **Simulation** — the mode without a robot: frames are printed, nobody is scanned (section 14, page 17).
- **Robot address** — its Bluetooth "name"; the remote remembers it, so `q` is quick (section 3, page 5).
- **STOP** — `space` or `Esc`: the robot freezes and drives no more (section 4, page 6).
- **To blink** — to flash the light briefly (the `b` key; section 5, page 7).
- **Diagram** — a picture in this manual; the sources live in `docs/diagrams/` (section 14, page 17).
- **Cruise control** — the mode where the robot drives on by itself until you press `space` (section 4, page 6).

# 16. Cheat sheet

**Remote:**

| Keys | What they do |
|---|---|
| `q` / `Shift+Q` | connect / disconnect |
| `w` `s` `a` `d`, arrows | drive (forward, backward, left, right) |
| `space` `Esc` | STOP |
| `]` `[` `+` `-` | speed up / down |
| `Shift` | boost (while held) |
| `1` … `8`, `0` | light colour, `0` turns it off |
| `b` `B` | blink |
| `v` `V` | motor 3: forward / backward (a switch) |
| `p` `P` | program mode |
| `Enter` | drive the program: the last macro or a file |
| `Shift+Enter` | the same, but without pauses |
| `r` `t` | drive the route back: backwards / turned around |
| `R` `T` | the same, but without pauses |
| `m` | record a macro (stop it with `Esc` or `n`) |
| `f` `g` | save / load a program file |
| `e` `E` | program editor (works from the remote too) |
| `y` `i` | test drive / motors one by one |
| `j` `k` | hold / cruise control |
| `h` `?` | help |
| `Ctrl+C` | quit |

**Program mode:** `w/a/s/d` — steps · digits — how many times · `space` — pause · `b` — blink · `c` — colour · `*` — repeat · `]` `[` — speed · `Backspace` — delete a step · `x` — clear · `Enter` — start (with `Shift` — without pauses) · `Esc` — to the remote. **The way back:** `r` `t` — backwards / turned around, `R` `T` — the same without pauses; works from the remote and from the editor.

**Run:** `space` — pause/resume · `Esc` — stop · `p` — edit. After a run: `Enter` — once more, `r` `t` — back.

A mini example (the square) as a file `programs/square.txt`:

```
speed 5
repeat 4
forward 4
right 2
blink 2
```

**In one line** (the most needed):

```
q connect · w a s d drive · space STOP · ] [ speed ·
1…8 colour · b blink · p program · Enter run · r t back ·
m macro · f g file · e editor · h help · Ctrl+C quit
```

**What to write in a program file:** `forward 4` · `back 2` · `left 1` · `right 2` · `pause 1` · `blink 2` · `color 3` · `repeat 4` · `speed 5` · `forward 1.5s`. The remote understands Russian words too: `вперёд`, `назад`, `влево`, `вправо`, `пауза`, `мигнуть`, `цвет 3`, `повтор 4`, `скорость 5`. Anything after `#` is a comment.

**If you forgot a key.** Press `h` — the remote shows the help. The `h` key works in program mode as well: there is a help of its own. And the run journal (the `--log` flag) shows what the robot did and how long it took.

**A mini program to memorise** (a square with light):

```
speed 5
repeat 4
forward 4
right 2
blink 2
```

How to run it: press `p`, type the steps (or load a file with `g`) and press `Enter`.

**Three useful habits:**

1. Before a run look at `[prog]`: how many steps and which speed.
2. After a run look at the `[run]` line: it shows how long it took.
3. Point the robot's nose away from you: then `w` means "forward".

**Where to go next.** The `examples/` folder holds ready programs (the square, the firefly and others) — press `g` and choose one. You can also race a friend: set up skittles and drive the track without knocking any down. The fastest one gets to command the robot!

**Three rescue keys:**

| Key | When it saves you |
|---|---|
| `Esc` | the robot is doing something wrong — an instant STOP |
| `q` | the link is lost — connect again |
| `Ctrl+C` | the remote is frozen — stop the motors and quit |

**Pilot check:**

| I can | ☐ |
|---|---|
| connect and disconnect (`q`, `Shift+Q`) | ☐ |
| drive, turn and change the speed | ☐ |
| light any colour and blink | ☐ |
| collect a program and run it | ☐ |
| record a macro and come back with `r` | ☐ |
| save a program and load it | ☐ |
| show the remote to a friend | ☐ |

> More programs and examples are in the `examples/` folder. A pilot who has made it this far: build your own track, put down a bottle as a target and try to drive around it without touching it. Good luck!